Document Number: P1188R0
Date: 2019-01-22
Audience: LEWG
Reply-To: Barry Revzin, barry dot revzin at gmail dot com

Library utilities for `<=>`

Contents

1. Introduction

In San Diego, I brought P1186R0 and P1187R0 to LEWG which did the following:

- Removed `std::compare_3way()`, the algorithm, and replaced it with a function object that invokes `<=>` on its operands
- Added a type trait, `std::compare_3way_type`, for the type of `<=>`

The motivation for removing `std::compare_3way()` was driven by P1186R0 redefining `<=>` in such a way that the algorithm itself would be completely pointless - `std::compare_3way(a, b)` would have become a long way of writing `a <=> b`. However, P1186 is changing course in a way that does not make the algorithm obsolete.

Since San Diego, I've also discovered the need for a new fundamental library tool for implementing `<=>`: we need a `concept`. This concept is very important, as I'll illustrate here.

To try to avoid confusion, instead of writing multiple R1s that refer to each other in a complex maze of indirections, I thought I would simply write one single paper that includes within it all the changes I am proposing for the Library for dealing with `<=>`. This is that paper.

2. Motivations and Proposals

This paper proposes two additions and one replacement to the library. I will go through them in increasing order of complexity.

2.1. Add a type trait: `std::compare_3way_type`

For some types, in order to implement `operator<=>()` you have to defer to the implementation of the comparison operators of other types. And more than that, you need to know what the comparison category is for those types you defer to in order to know what comparison category to return. For example, to implement `<=>` for a type like `optional<T>`:

```
template <typename T, typename U>
constexpr ??? operator<=>(optional<T> const& lhs, optional<U> const& rhs) {
    if (lhs && rhs) {
        return *lhs <=> *rhs;
    } else {
        return lhs.has_value() <=> rhs.has_value();
    }
}
```

What do we put in the `???`? We can't put `auto`, because our two `return` statements might have different types; the `bool` comparison has type `strong_ordering` but the other one could have any of the five comparisons. Whichever comparison category `*lhs <=> *rhs` has, `strong_ordering` will be convertible to it, so we can simply use that expression directly:

```
template <typename T, typename U>
constexpr auto operator<=>(optional<T> const& lhs, optional<U> const& rhs)
    -> decltype(*lhs <=> *rhs)
{ /* ... */ }
```

This will come up basically every time you need to defer to a template parameter:

```
template <typename T, typename U>
auto operator<=>(vector<T> const& lhs, vector<U> const& rhs)
    -> decltype(lhs[0] <=> rhs[0]);

template <typename T1, typename E1, typename T2, typename E2>
auto operator<=>(expected<T1,E1> const& lhs, expected<T2,E2> const& rhs)
    -> common_comparison_category_t<
            decltype(lhs.value() <=> rhs.value()),
            decltype(lhs.error() <=> rhs.error())>;
```

We see the same thing over and over and over again. We need to know what comparison category to return, and this comparison category is a property of the types that we're comparing.

That's a type trait. A type trait that's currently missing from the standard library:

```
template<class T, class U = T> struct compare_3way_type;

template<class T, class U = T>
using compare_3way_type_t = typename compare_3way_type<T, U>::type;
```

Such that `compare_3way_type<T, U>` has a member `type` that is `decltype(declval<CREF<T>>() <=> declval<CREF<U>>())` is that is a valid expression, and no member `type` otherwise (where `CREF<T>` is `remove_reference_t<T> const&`).

Today	Proposed

`template <typename T, typename U>`
`constexpr auto`
`operator<=>(optional<T> const& lhs, optional<U> const& rhs)`
`    -> decltype(*lhs <=> *rhs);`
	`template <typename T, typename U>`
	`constexpr compare_3way_type_t<T, U>`
	`operator<=>(optional<T> const& lhs, optional<U> const& rhs);`

2.2. Add a pair of concepts: `ThreeWayComparable` and `ThreeWayComparableWith`

One important piece of functionality that libraries will need to implement is to *conditionally* provide `<=>` on class templates based on whether certain types provide `<=>`. For example. `vector<T>` should expose an `operator<=>` if and only if `T` has an `operator<=>`.

Coming up with the correct way to implement this is non-trivial - I've gone through several incorrect implementations of it between the time when I started writing P1186R0 and now. The key problem to be solved is to ensure that `<` invokes `<=>` if at all possible. That is:

```
struct A {
    bool operator<(A const&) const;
};

struct B {
    strong_ordering operator<=>(B const&) const;
}

template <typename T> vector<T> get();

get<A>() < get<A>(); // should call vector<A>::operator<
get<B>() < get<B>(); // should call vector<B>::operator<=>
```

The best way to solve this problem is with the use of concepts. We need to make `operator<=>` *more constrained* than `operator<` (which might mean carefully ensuring subsumption, or just adding a constraint at all). This implementation strategy makes it possible to provide a complete implementation of conditional `<=>` for `vector<T>` as follows:

```
// uncontrained ==, != is not necessary after P1185
template <typename T> bool operator==(vector<T> const&, vector<T> const&);

// unconstrained <,>,<=,>=
template <typename T> bool operator<(vector<T> const&, vector<T> const&);
template <typename T> bool operator>(vector<T> const&, vector<T> const&);
template <typename T> bool operator<=(vector<T> const&, vector<T> const&);
template <typename T> bool operator>=(vector<T> const&, vector<T> const&);

template <ThreeWayComparable T>  // this constraint is critical
compare_3way_type_t<T>           // from §2.1
operator<=>(vector<T> const&, vector<T> const&);
```

This works because in the expression `get<B>() < get<B>()`, both `operator<` and `operator<=>` are candidates with the same conversion sequences. The relevant order of tiebreakers in [over.match.best] is:

1.6. Prefer non-template to template
1.7. Prefer more specialized template
1.8. Prefer more constrained template
1.9. Prefer derived constructor to inherited base constructor
1.10. Prefer non-rewritten-candidate (i.e. `<`) to rewritten candidate (i.e. `<=>`)
1.11. Prefer non-reversed rewritten candidate to reversed rewritten candidate

Importantly, preferring the *more constrained* candidate is an earlier tiebreaker than preferring the direct relational operator to spaceship. This is the rule that ensures that we can prefer `<=>` by just making that operator function more constrained than any of the relational operators (which is trivial if none of them are constrained, as above).

There are other ways of ensuring `<=>` gets invoked over `<`, but I believe this to be the best way since it only requires properly declaring `<=>`.

And this way requires a concept for `<=>`. This kind of concept is fairly fundamental, and because of its wide application and its subtle complexity, this paper proposes it be included in the standard library. Otherwise, everyone will have to write their own slightly different, potentially incorrect version of it. Due to concept subsumption, there is especial value of having a common understanding of what `ThreeWayComparable` means.

With help from Casey Carter, the implementation I'm proposing is (again using `CREF<T>` as alias for `remove_reference_t<T> const&`):

```
template <typename T, typename Cat>
  concept compares-as = // exposition only
    Same<common_comparison_category_t<T, Cat>, Cat>;

template <typename T, typename Cat=std::weak_equality>
  concept ThreeWayComparable = requires(CREF<T> a, CREF<T> b) {
    { a <=> b } -> compares-as<Cat>;
  };

template <typename T, typename U,
          typename Cat=std::weak_equality>
  concept ThreeWayComparableWith =
    ThreeWayComparable<T, Cat> &&
    ThreeWayComparable<U, Cat> &&
    CommonReference<CREF<T>, CREF<U>> &&
    ThreeWayComparable<
      common_reference_t<CREF<T>, CREF<U>>,
      Cat> &&
    requires(CREF<T> t, CREF<U> u) {
      { t <=> u } -> compares-as<Cat>;
      { u <=> t } -> compares-as<Cat>;
    };
```

This definition follows the practice of `EqualityComparable` and `StrictTotallyOrdered` in that we're not just checking syntactic correctness - we're doing a bit more than that: we're also requiring that `<=>` actually produces one of the comparison categories.

The defaulted `Cat` parameter allows users to refine their constraints. `ThreeWayComparable<T>` just requires that `T` provide some meaningful `<=>`, whereas `ThreeWayComparable<T, std::partial_ordering>` requires that `T` provide an ordering.

Note that `ThreeWayComparable<T, std::strong_ordering>` does *not* subsume `ThreeWayComparable<T, std::weak_ordering>` despite being logically stricter.

2.3. Replace `std::compare_3way()`

While P1186R0 made `std::compare_3way()` completely pointless, that is no longer strictly the case with P1186R1. Nevertheless, this paper proposes removing that algorithm.

It is very easy to misuse without thinking about it because it automatically gives you a strong ordering - which isn't apparent from the name. It's very easy to fall into the trap of thinking that this is just the correct algorithm to use whenever you want a three-way comparison, and it's really not. Lawrence Crowl in P1380R0 argues that the comparisons it synthesizes should be `weak_XXX` instead of `strong_XXX`, but even then it still means the library has to make one singular choice for what the correct comparison category to synthesize is. Which is the right default? Is there a right default?

I think it would be strictly superior to use the `compare_3way_fallback<strong_ordering>` function as described in P1186R1, which really makes clear what is going on. It's easy to implement on the back of the language changed proposed in that paper, and it allows for a more powerful algorithm that is also more correct than `std::compare_3way()`. So let's remove `std::compare_3way()`.

On the other hand, having a function object that just invokes `<=>` would actually be useful, and `compare_3way` seems like the best name for it.

The question is: what should `compare_3way`, the function object that invokes `<=>`, look like? The model we have is `std::less`, where `std::less<T>` compares two objects of type `T` and `std::less<void>` deduces its arguments. But we also recently acquired `std::ranges::less`, which at the moment mirrors the `std::less` version except additionally using constrains. But P1252 proposes to simplify to just be a single transparent comparison.

The simplest version of a spaceship comparison function object would be to take the P1252 version:

```
namespace std {
  struct compare_3way {
    template<class T, class U>
      requires ThreeWayComparableWith<T, U> // §2.2
               || BUILTIN_PTR_3WAY(T, U)     // see [range.cmp] for inspiration
    constexpr auto operator()(T&& t, U&& u) const;

    using is_transparent = unspecified;
  };
}
```

This is inconsistent with `std::less`, since it's not a template of any kind and there's no "fixed-type" version. But it's also a lot simpler. The question for LEWG is do they prefer the above or if they want to fully copy `std::less`, which as it stands in the working draft today would be:

```
namespace std {
  template<class T = void>
  struct compare_3way {
    constexpr auto operator()(const T&, const T&) const;
  };

  template <> struct compare_3way<void> {
    template<class T, class U>
    constexpr auto operator()(T&& t, U&& u) const
      -> decltype(std::forward<T>(t) <=> std::forward<U>(u));
  };

  namespace ranges {
    template<class T = void>
      requires ThreeWayComparable<T> || Same<T, void> || BUILTIN_PTR_3WAY(const T&, const T&)
    struct compare_3way {
      constexpr auto operator()(const T& x, const T& y) const;
    };

    template<> struct compare_3way<void> {
      template<class T, class U>
        requires ThreeWayComparableWith<T, U> || BUILTIN_PTR_3WAY(T, U)
      constexpr auto operator()(T&& t, U&& u) const;
```

```
      using is_transparent = unspecified;
    };
  }
}
```

3. Wording

Add the new trait, concept, and function object into the `<compare>` synopsis in 16.11.1 [compare.syn]:

```
namespace std {
  [...]

  // [cmp.common], common comparison category type
  template<class... Ts>
  struct common_comparison_category {
    using type = see below;
  };
  template<class... Ts>
    using common_comparison_category_t = typename common_comparison_category<Ts...>::type;

  // [cmp.threewaycomparable], concept ThreeWayComparable
  template<class T, class Cat = weak_equality>
    concept ThreeWayComparable = see below;
  template<class T, class U, class Cat = weak_equality>
    concept ThreeWayComparableWith = see below;

  // [cmp.3way], compare_3way
  template<class T, class U = T> struct compare_3way_type;

  template<class T, class U = T>
    using compare_3way_type_t = typename compare_3way_type<T, U>::type;

  template<class T = void> struct compare_3way;
  template<> struct compare_3way<void>;

  namespace ranges {
    // [???] concept-constrained comparisons
    template<class T = void>
      requires see below
    struct compare_3way;

    template<> struct compare_3way<void>;
  }
  [...]
}
```

Add a new clause [cmp.threewaycomparable]. We don't need to add any new semantic constraints. The requirement that the `<=>` s used have to be equality-preserving is picked up through [concepts.equality] already.

```
template <typename T, typename Cat>
  concept compares-as = // exposition only
    Same<common_comparison_category_t<T, Cat>, Cat>;

template <typename T, typename Cat=std::weak_equality>
  concept ThreeWayComparable =
    requires(const remove_reference_t<T>& a,
             const remove_reference_t<T>& b) {
      { a <=> b } -> compares-as<Cat>;
    };

template <typename T, typename U,
          typename Cat=std::weak_equality>
  concept ThreeWayComparableWith =
    ThreeWayComparable<T, Cat> &&
    ThreeWayComparable<U, Cat> &&
    CommonReference<const remove_reference_t<T>&, const remove_reference_t<U>&> &&
```

```
    ThreeWayComparable<
      common_reference_t<const remove_reference_t<T>&, const remove_reference_t<U>&>,
      Cat> &&
    requires(const remove_reference_t<T>& t,
             const remove_reference_t<U>& u) {
      { t <=> u } -> compares-as<Cat>;
      { u <=> t } -> compares-as<Cat>;
    };
```

Add a new specification for `compare_3way_type` in a new clause after 16.11.3 [cmp.common] named [cmp.3way]:

> The behavior of a program that adds specializations for the `compare_3way_type` template defined in this subclause is undefined.
>
> For the `compare_3way_type` type trait applied to the types `T` and `U`, let `t` and `u` denote lvalues of types `const remove_reference_t<T>` and `const remove_reference_t<U>`. If the expression `t <=> u` is well formed, the member *typedef-name* `type` shall equal `decltype(t <=> u)`. Otherwise, there shall be no member `type`.

Add a specification for `compare_3way` to a new clause after 16.11.3 [cmp.common] named [cmp.3way]:

> The specializations of `compare_3way` for any pointer type yield a strict total order that is consistent among those specializations and is also consistent with the partial order imposed by the built-in operator `<=>`. For the template specialization `compare_3way<void>`, if the call operator calls a built-in operator comparing pointers, the call operator yields a strict total order that is consistent among those specializations and is also consistent with the partial order imposed by that built-in operator.
>
> ```
> template<class T = void> struct compare_3way {
>   constexpr auto operator()(const T& x, const T& y) const;
> };
> ```
>
> `constexpr auto operator()(const T& x, const T& y) const;`
> *Returns*: `x <=> y`.
>
> ```
> template<> struct compare_3way<void> {
>   template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
>     -> decltype(std::forward<T>(t) <=> std::forward<U>(u));
>
>   using is_transparent = unspecified;
> };
> ```
>
> ```
> template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
>   -> decltype(std::forward<T>(t) <=> std::forward<U>(u));
> ```
>
> *Returns*: `std::forward<T>(t) <=> std::forward<U>(u)`.

Add the following wording somewhere for `std::ranges::compare_3way`, the equivalent of `std::ranges::less` for `std::compare_3way` and is mostly copied from [range.cmp]. I don't know where it needs to, or how we want to arrange it.

> In this subclause, `BUILTIN_PTR_3WAY(T, U)` for types `T` and `U` is a boolean constant expression. `BUILTIN_PTR_3WAY(T, U)` is `true` if and only if `<=>` in the expression `declval<T>() <=> declval<U>()` resolves to a built-in operator comparing pointers.
>
> There is an implementation-defined strict total ordering over all pointer values of a given type. This total ordering is consistent with the partial order imposed by the builtin operator `<=>`.
>
> ```
> template<class T = void>
>   requires ThreeWayComparable<T> || Same<T, void> || BUILTIN_PTR_3WAY(const T&, const T&)
> struct ranges::compare_3way {
>   constexpr auto operator()(const T& x, const T& y) const;
> };
> ```
>
> `operator()` has effects equivalent to: `return ranges::compare_3way<>{}(x, y);`

```
template<> struct ranges::compare_3way<void> {
  template<class T, class U>
    requires ThreeWayComparableWith<T, U> || BUILTIN_PTR_3WAY(T, U)
  constexpr auto operator()(T&& t, U&& u) const;

  using is_transparent = unspecified;
};
```

Expects: If the expression `std::forward<T>(t) <=> std::forward<U>(u)` results in a call to a built-in operator `<=>` comparing pointers of type `P`, the conversion sequences from both `T` and `U` to `P` shall be equality-preserving ([concepts.equality]).

Effects:

- If the expression `std::forward<T>(t) <=> std::forward<U>(u)` results in a call to a built-in operator `<=>` comparing pointers of type `P`: returns `strong_ordering::less` if (the converted value of) `t` precedes `u` in the implementation-defined strict total order over pointers of type `P`, `strong_ordering::greater` if `u` precedes `t`, and otherwise `strong_ordering::equal`.
- Otherwise, equivalent to: `return std::forward<T>(t) <=> std::forward<U>(u);`

Remove `std::compare_3way()` from synopsis in 23.4 [algorithm.syn]:

```
namespace std {
  [...]
  // [alg.3way], three-way comparison algorithms
  ~~template<class T, class U>~~
  ~~  constexpr auto compare_3way(const T& a, const U& b);~~
  template<class InputIterator1, class InputIterator2, class Cmp>
    constexpr auto
      lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
                                   InputIterator2 b2, InputIterator2 e2,
                                   Cmp comp)
        -> common_comparison_category_t<decltype(comp(*b1, *b2)), strong_ordering>;
  template<class InputIterator1, class InputIterator2>
    constexpr auto
      lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
                                   InputIterator2 b2, InputIterator2 e2);
  [...]
}
```

Remove the specification of `std::compare_3way()` of 23.7.11 [alg.3way]:

~~`template<class T, class U> constexpr auto compare_3way(const T& a, const U& b);`~~

~~*Effects*: Compares two values and produces a result of the strongest applicable comparison category type:~~

- ~~Returns a <=> b if that expression is well-formed.~~
- ~~Otherwise, if the expressions a == b and a < b are each well-formed and convertible to bool, returns strong_ordering::equal when a == b is true, otherwise returns strong_ordering::less when a < b is true, and otherwise returns strong_ordering::greater.~~
- ~~Otherwise, if the expression a == b is well-formed and convertible to bool, returns strong_equality::equal when a == b is true, and otherwise returns strong_equality::nonequal.~~
- ~~Otherwise, the function is defined as deleted.~~

Change the specification of `std::lexicographical_compare_3way` in 23.7.11 [alg.3way] paragraph 4:

```
template<class InputIterator1, class InputIterator2>
  constexpr auto
    lexicographical_compare_3way(InputIterator1 b1, InputIterator1 e1,
                                 InputIterator2 b2, InputIterator2 e2);
```

Effects: Equivalent to:

```
return lexicographical_compare_3way(b1, e1, b2, e2, compare_3way());
                                    ~~[](const auto& t, const auto& u) {~~
                                    ~~  return compare_3way(t, u);~~
                                    ~~});~~
```

3.1. Alternate Wording for `std::compare_3way`

If LEWG decides on just the single, non-template `std::compare_3way` function object, the wording can be as follows.

In 16.11.1 [compare.syn]:

```
namespace std {
  [...]

  // [cmp.common], common comparison category type
  template<class... Ts>
  struct common_comparison_category {
    using type = see below;
  };
  template<class... Ts>
    using common_comparison_category_t = typename common_comparison_category<Ts...>::type;

  // [cmp.threewaycomparable], concept ThreeWayComparable
  template<class T, class Cat = weak_equality>
    concept ThreeWayComparable = see below;
  template<class T, class U, class Cat = weak_equality>
    concept ThreeWayComparableWith = see below;

  // [cmp.3way], compare_3way
  template<class T, class U = T> struct compare_3way_type;

  template<class T, class U = T>
    using compare_3way_type_t = typename compare_3way_type<T, U>::type;

  struct compare_3way;
  [...]
}
```

In the new subclause for `std::compare_3way`:

> In this subclause, `BUILTIN_PTR_3WAY(T, U)` for types `T` and `U` is a boolean constant expression. `BUILTIN_PTR_3WAY(T, U)` is `true` if and only if `<=>` in the expression `declval<T>() <=> declval<U>()` resolves to a built-in operator comparing pointers.
>
> There is an implementation-defined strict total ordering over all pointer values of a given type. This total ordering is consistent with the partial order imposed by the builtin operator `<=>`.
>
> ```
> struct compare_3way {
>   template<class T, class U>
>     requires ThreeWayComparableWith<T,U> || BUILTIN_PTR_3WAY(T, U)
>   constexpr auto operator()(T&& t, U&&u) const;
>
>   using is_transparent = unspecified;
> };
> ```
>
> *Expects*: If the expression `std::forward<T>(t) <=> std::forward<U>(u)` results in a call to a built-in operator `<=>` comparing pointers of type `P`, the conversion sequences from both `T` and `U` to `P` shall be equality-preserving ([concepts.equality]).
>
> *Effects*:
>
> - If the expression `std::forward<T>(t) <=> std::forward<U>(u)` results in a call to a built-in operator `<=>` comparing pointers of type `P`: returns `strong_ordering::less` if (the converted value of) `t` precedes `u` in the implementation-defined strict total order over pointers of type `P`, `strong_ordering::greater` if `u` precedes `t`, and otherwise `strong_ordering::equal`.
> - Otherwise, equivalent to: `return std::forward<T>(t) <=> std::forward<U>(u);`

4. Acknowledgments

Thank you to Agustín Bergé, Casey Carter and Tim Song for many extensive conversations around these issues.

5. References

- [P1186R0] "When do you actually use <=>?" by Barry Revzin, 2018-10-07
- [P1187R0] "A type trait for std::compare_3way()'s type" by Barry Revzin, 2018-10-07
- [P1252R0] "Ranges Design Cleanup" by Casey Carter, 2018-10-07
- [P1380R0] "Ambiguity and Insecurities with Three-Way Comparison" by Lawrence Crowl, 2018-11-26